Promise 从入门到自定义

一、Promise 的理解和使用

1、Promise 是什么?

①、理解

Ⅰ、抽象表达:

Ⅱ、具体表达:

②、promise 的状态改变

③、promise 的基本流程

2、为什么要用 Promise

①、指定回调函数的方式更加灵活

②、支持链式调用, 可以解决回调地狱问题

③、

④、

3、如何使用 Promise?

①、API

Ⅰ、Promise 构造函数: Promise (excutor) {}

Ⅱ、Promise.prototype.then 方法: (onResolved, onRejected) => {}

Ⅲ、Promise.prototype.catch 方法: (onRejected) => {}

Ⅳ、Promise.resolve 方法: (value) => {}

Ⅴ、Promise.reject 方法: (reason) => {}

Ⅵ、Promise.all 方法: (promises) => {}

Ⅶ、Promise.race 方法: (promises) => {}

②、

4、promise 的基本使用

二、实践练习

1、fs 读取文件

①、创建 resource 文件夹，其中创建 content.txt

②、回调函数 形式

③、Promise 形式

2、

3、

4、

三、

四、

五、

1、

2、

3、

4、

5、

6、

7、

8、

9、

①、

②、

③、

④、

⑤、

⑥、

⑦、

⑧、

⑨、

⑩、

Ⅰ、

Ⅱ、

Ⅲ、

Ⅳ、

Ⅴ、

Ⅵ、

Ⅶ、

Ⅷ、

Ⅸ、

Ⅹ、

> https://www.bilibili.com/video/BV1GA411x7z1?p=5&spm_id_from=pageDriver&vd_source=b55e15966ca689b32671d4aa387cab01
>
> 📎尚硅谷_Promise从入门到自定义.pdf
>
> 📎promise.jpg

一、Promise 的理解和使用

1、Promise 是什么?

①、理解

Ⅰ、抽象表达:

1. Promise 是一门新的技术(ES6 规范)
2. Promise 是 JS 中进行异步编程的新解决方案
3. 备注：旧方案是单纯使用回调函数

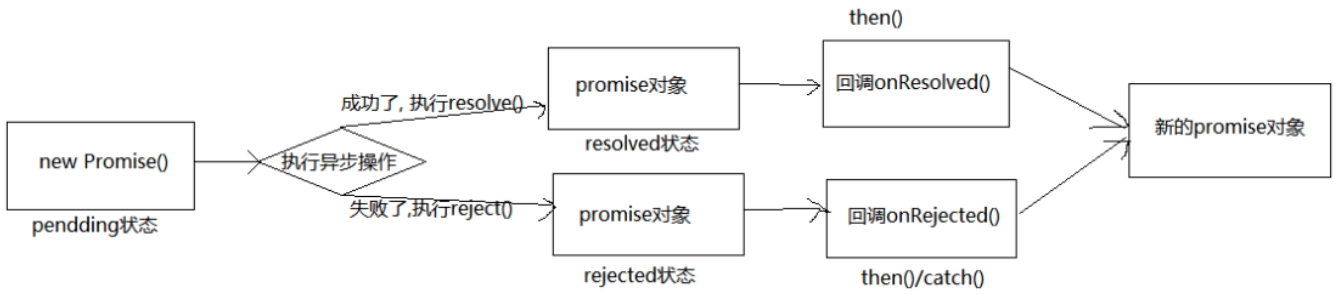
Ⅱ、具体表达:

1. 从语法上来说: Promise 是一个构造函数
2. 从功能上来说: promise 对象用来封装一个异步操作并可以获取其成功/失败的结果值

②、promise 的状态改变

1. pending 变为 resolved
2. pending 变为 rejected
3. 说明: 只有这 2 种, 且一个 promise 对象只能改变一次。无论变为成功还是失败, 都会有一个结果数据
4. 成功的结果数据一般称为 value, 失败的结果数据一般称为 reason

③、promise 的基本流程



2、为什么要用 Promise

①、指定回调函数的方式更加灵活

1. 旧的: 必须在启动异步任务前指定
2. promise: 启动异步任务 => 返回promie对象 => 给promise对象绑定回调函数(甚至可以在异步任务结束后指定/多个)

②、支持链式调用, 可以解决回调地狱问题

1. 什么是回调地狱?
    a. 回调函数嵌套调用, 外部回调函数异步执行的结果是嵌套的回调执行的条件
2. 回调地狱的缺点?
    a. 不便于阅读
    b. 不便于异常处理
3. 解决方案?
    a. promise 链式调用
4. 终极解决方案
    a. async/await

③、

④、

3、如何使用 Promise?

①、API

Ⅰ、`Promise 构造函数: Promise (excutor) {}`

1. executor 函数: 执行器 (resolve, reject) => {}
2. resolve 函数: 内部定义成功时我们调用的函数 value => {}
3. reject 函数: 内部定义失败时我们调用的函数 reason => {}
4. 说明: executor 会在 Promise 内部立即同步调用,异步操作在执行器中执行

Ⅱ、`Promise.prototype.then 方法: (onResolved, onRejected) => {}`

1. onResolved 函数: 成功的回调函数 (value) => {}
2. onRejected 函数: 失败的回调函数 (reason) => {}
3. 说明: 指定用于得到成功 value 的成功回调和用于得到失败 reason 的失败回调，返回一个新的 promise 对象

Ⅲ、`Promise.prototype.catch 方法: (onRejected) => {}`

1. onRejected 函数: 失败的回调函数 (reason) => {}
2. 说明: then()的语法糖, 相当于: then(undefined, onRejected)

Ⅳ、`Promise.resolve 方法: (value) => {}`

1. value: 成功的数据或 promise 对象
2. 说明: 返回一个成功/失败的 promise 对象

Ⅴ、`Promise.reject 方法: (reason) => {}`

1. reason: 失败的原因
2. 说明: 返回一个失败的 promise 对象

Ⅵ、`Promise.all 方法: (promises) => {}`

1. promises: 包含 n 个 promise 的数组
2. 说明: 返回一个新的 promise, 只有所有的 promise 都成功才成功, 只要有一个失败了就直接失败

Ⅶ、`Promise.race 方法: (promises) => {}`

1. promises: 包含 n 个 promise 的数组
2. 说明: 返回一个新的 promise, 第一个完成的 promise 的结果状态就是最终的结果状态

②、

4、promise 的基本使用

1. 使用以前的方式

HTML | 复制代码

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link crossorigin='anonymous' href="https://cdn.bootcss.com/twitter-bo
otstrap/3.3.7/css/bootstrap.min.css" rel="stylesheet">

</head>
<body>
    <div class="container">…</div>

</body>

    <script>
        //生成随机数
        function rand(m,n){
            return Math.ceil(Math.random() * (n-m+1)) + m-1;
        }

        // 获取元素对象
        const btn = document.querySelector("#btn")
        // 绑定单击事件
        btn.addEventListener("click",function(){

            // 以前的方式
            // 定时器，1 秒后响应
            setTimeout(() => {
                // 生成 1-100 的随机数
                let n = rand(1, 100);
                // 判断
                if(n <= 30){
                    alert("中奖")
                }else{
                    alert("未中奖")
                }
            },1000)

        })
    </script>

</html>
```

2. 使用 promise

HTML | 复制代码

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link crossorigin='anonymous' href="https://cdn.bootcss.com/twitter-bo
otstrap/3.3.7/css/bootstrap.min.css" rel="stylesheet">

</head>
<body>
    <div class="container">
        <h2>promise 初体验</h2>
        <button class="btn" id="btn">点击抽奖</button>
    </div>

</body>

    <script>
        //生成随机数
        function rand(m,n){
            return Math.ceil(Math.random() * (n-m+1)) + m-1;
        }

        // 获取元素对象
        const btn = document.querySelector("#btn")
        // 绑定单击事件
        btn.addEventListener("click",function(){

            // promise
            // resolve: 解决，函数类型的数据，异步任务成功调用他
            // reject:  拒绝，函数类型的数据，异步任务失败调用他
            const p = new Promise((resolve,reject) => {
                // 定时器，1 秒后响应
                setTimeout(() => {
                    // 生成 1-100 的随机数
                    let n = rand(1, 100);
                    // 判断
                    if(n <= 30){
                        // 将 promise 的状态设置为成功
                        resolve();
                    }else{
                        // 将 promise 的状态设置为失败
                        reject();
```

```
                    }
                },1000)
            })

            // 调用 then 方法
            // 需要两个参数，都是函数
            // 参数1：promise 成功时执行的函数
            // 参数2：promise 失败时执行的函数
            p.then(() =>{
                alert("中奖")
            }, () => {
                alert("未中奖")
            })

        })
    </script>

</html>
```

3. 使用 promise 传递结果（参数）

HTML | 复制代码

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <link crossorigin='anonymous' href="https://cdn.bootcss.com/twitter-bo
otstrap/3.3.7/css/bootstrap.min.css" rel="stylesheet">

</head>
<body>
    <div class="container">
        <h2>promise 初体验</h2>
        <button class="btn" id="btn">点击抽奖</button>
    </div>

</body>

    <script>
        //生成随机数
        function rand(m,n){
            return Math.ceil(Math.random() * (n-m+1)) + m-1;
        }

        // 获取元素对象
        const btn = document.querySelector("#btn")
        // 绑定单击事件
        btn.addEventListener("click",function(){

            // promise
            // resolve: 解决，函数类型的数据，异步任务成功调用他
            // reject:  拒绝，函数类型的数据，异步任务失败调用他
            const p = new Promise((resolve,reject) => {
                // 定时器，1 秒后响应
                setTimeout(() => {
                    // 生成 1-100 的随机数
                    let n = rand(1, 100);
                    // 判断
                    if(n <= 30){
                        // 将 promise 的状态设置为成功
                        resolve(n);
                    }else{
                        // 将 promise 的状态设置为失败
                        reject(n);
```

```
                    }
                },1000)
            })

            // 调用 then 方法
            // 需要两个参数，都是函数
            // 参数1：promise 成功时执行的函数；value：值
            // 参数2：promise 失败时执行的函数；reason：理由
            p.then((value) =>{
                alert("中奖，抽到的数字为：" + value)
            }, (reason) => {
                alert("未中奖，抽到的数字为：" + reason)
            })

        })
    </script>

</html>
```

二、实践练习

1、fs 读取文件

①、创建 resource 文件夹，其中创建 content.txt

HTML | 复制代码

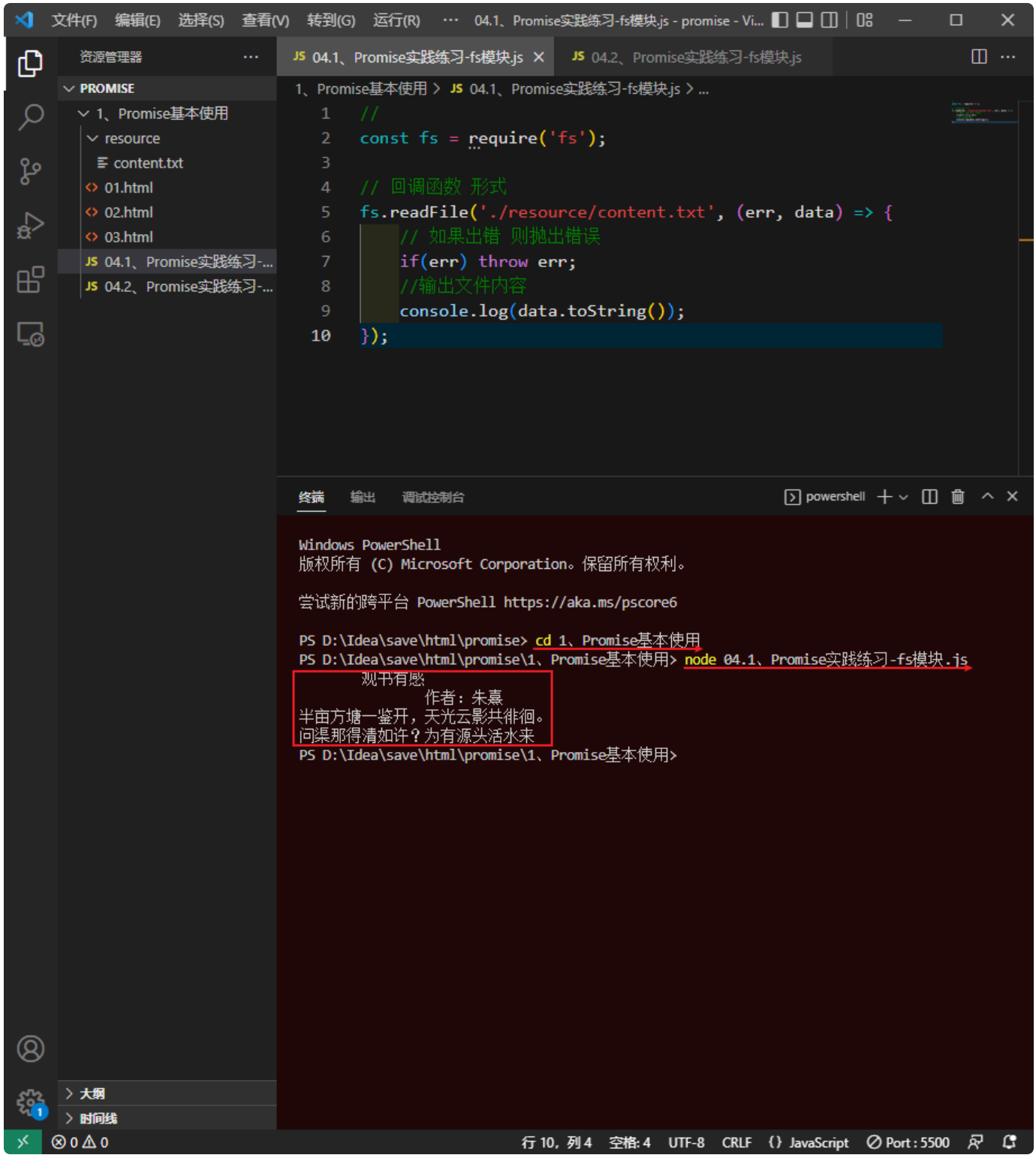
```
            观书有感
                    作者：朱熹
半亩方塘一鉴开，天光云影共徘徊。
问渠那得清如许？为有源头活水来
```

②、回调函数 形式

```javascript
//
const fs = require('fs');

// 回调函数 形式
fs.readFile('./resource/content.txt', (err, data) => {
    // 如果出错 则抛出错误
    if(err) throw err;
    //输出文件内容
    console.log(data.toString());
});
```

- 在控制台运行


文件(F) 编辑(E) 选择(S) 查看(V) 转到(G) 运行(R) ··· 04.1、Promise实践练习-fs模块.js - promise - Vi...
资源管理器
PROMISE
1、Promise基本使用
resource
content.txt
01.html
02.html
03.html
04.1、Promise实践练习-...
04.2、Promise实践练习-...
04.1、Promise实践练习-fs模块.js
04.2、Promise实践练习-fs模块.js
1、Promise基本使用 > 04.1、Promise实践练习-fs模块.js > ...
//
const fs = require('fs');
// 回调函数 形式
fs.readFile('./resource/content.txt', (err, data) => {
// 如果出错 则抛出错误
if(err) throw err;
//输出文件内容
console.log(data.toString());
});
终端 输出 调试控制台
powershell
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。
尝试新的跨平台 PowerShell https://aka.ms/pscore6
PS D:\Idea\save\html\promise> cd 1、Promise基本使用
PS D:\Idea\save\html\promise\1、Promise基本使用> node 04.1、Promise实践练习-fs模块.js
观书有感
作者：朱熹
半亩方塘一鉴开，天光云影共徘徊。
问渠那得清如许？为有源头活水来
PS D:\Idea\save\html\promise\1、Promise基本使用>
大纲
时间线
行 10，列 4 空格: 4 UTF-8 CRLF {} JavaScript Port : 5500

③、Promise 形式

JavaScript | 复制代码

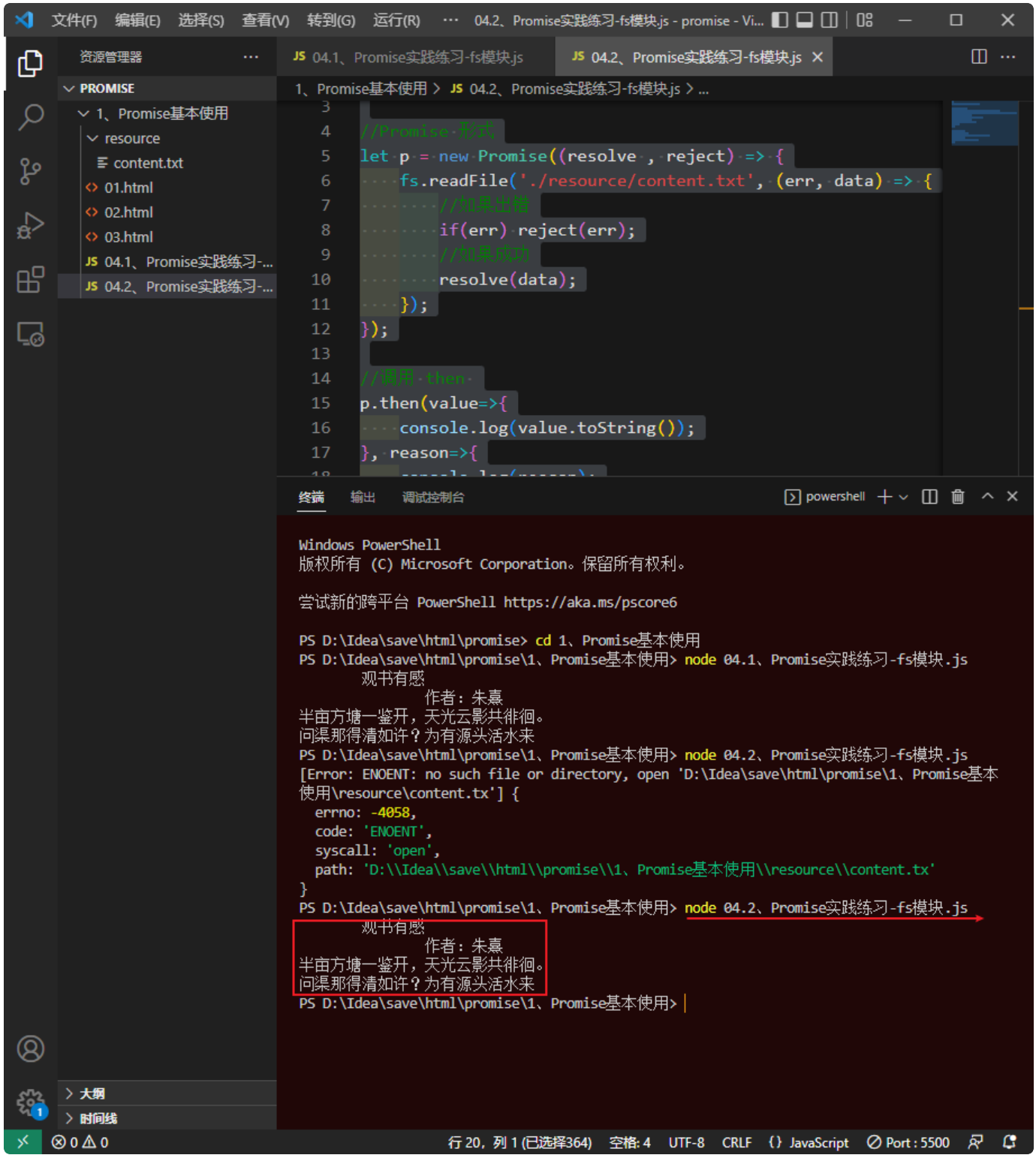
```javascript
//
const fs = require('fs');

//Promise 形式
let p = new Promise((resolve , reject) => {
    fs.readFile('./resource/content.txt', (err, data) => {
        //如果出错
        if(err) reject(err);
        //如果成功
        resolve(data);
    });
});

//调用 then
p.then(value=>{
    console.log(value.toString());
}, reason=>{
    console.log(reason);
});

```

- 在控制台运行


文件(F) 编辑(E) 选择(S) 查看(V) 转到(G) 运行(R) ··· 04.2、Promise实践练习-fs模块.js - promise - Vi...
资源管理器
PROMISE
1、Promise基本使用
resource
content.txt
01.html
02.html
03.html
04.1、Promise实践练习-...
04.2、Promise实践练习-...
04.1、Promise实践练习-fs模块.js
04.2、Promise实践练习-fs模块.js
1、Promise基本使用 > 04.2、Promise实践练习-fs模块.js > ...
//Promise 形式
let p = new Promise((resolve , reject) => {
fs.readFile('./resource/content.txt', (err, data) => {
//加果出错
if(err) reject(err);
//加果成功
resolve(data);
});
});
//调用 then
p.then(value=>{
console.log(value.toString());
}, reason=>{
终端 输出 调试控制台
powershell
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。
尝试新的跨平台 PowerShell https://aka.ms/pscore6
PS D:\Idea\save\html\promise> cd 1、Promise基本使用
PS D:\Idea\save\html\promise\1、Promise基本使用> node 04.1、Promise实践练习-fs模块.js
观书有感
作者：朱熹
半亩方塘一鉴开，天光云影共徘徊。
问渠那得清如许？为有源头活水来
PS D:\Idea\save\html\promise\1、Promise基本使用> node 04.2、Promise实践练习-fs模块.js
[Error: ENOENT: no such file or directory, open 'D:\Idea\save\html\promise\1、Promise基本使用\resource\content.tx'] {
errno: -4058,
code: 'ENOENT',
syscall: 'open',
path: 'D:\\Idea\\save\\html\\promise\\1、Promise基本使用\\resource\\content.tx'
}
PS D:\Idea\save\html\promise\1、Promise基本使用> node 04.2、Promise实践练习-fs模块.js
观书有感
作者：朱熹
半亩方塘一鉴开，天光云影共徘徊。
问渠那得清如许？为有源头活水来
PS D:\Idea\save\html\promise\1、Promise基本使用>
大纲
时间线
⊗ 0 ⚠ 0
行 20，列 1 (已选择364) 空格: 4 UTF-8 CRLF {} JavaScript Port : 5500

2、

3、

4、

三、

四、

五、

1、

2、

3、

4、

5、

6、

7、

8、

9、

①、

②、

③、

④、

⑤、

⑥、

⑦、

⑧、

⑨、

⑩、

Ⅰ、

Ⅱ、

Ⅲ、

Ⅳ、

Ⅴ、

Ⅵ、

Ⅶ、

Ⅷ、

Ⅸ、

Ⅹ、